Universidad Nacional de Loja

FACULTAD DE ENERGÍA LAS INDUSTRIAS Y
LOS RECURSOS NATURALES NO RENOVABLES

AA (Semana dos)
Investigación Persistencia de Datos

Nombres y Apellidos:

Paolo Julián Merino Guamán

Carrera:

Ing. en Computación

Ciclo:

3er Ciclo

Docente:

Ing. Cristian Narváez

Fecha:

11 de Octubre del 2026

LATENCIA DE ALMACENAMIENTO Y SOFTWARE
SISTEMAS DISTRIBUIDOS

1. Jerarquía de Memoria y Brecha de Desempeño Físico

La velocidad de respuesta del software se ve fuertemente limitada por la latencia de almacenamiento cuando se requiere persistencia de datos. Esta brecha de rendimiento radica en la propia arquitectura y distribución física del hardware [2]:

- **RAM vs. Discos:** La memoria RAM es aproximadamente 100.000 veces más rápida que el almacenamiento secundario.
- **Mecánicos (HDD):** Su latencia es alta ($\sim$47,85 ms en escrituras 4K) debido al tiempo físico que toma mover los platos magnéticos y el cabezal.
- **Estado Sólido (SSD y NVMe):** Carecen de partes mecánicas. Al usar chips Flash y conectarse directo al bus PCIe, procesan colas en paralelo, reduciendo la latencia a microsegundos ($\sim$145,53 μs) y multiplicando el ancho de banda.

2. Amplificación de la Latencia por Software

Las garantías y el diseño de la pila de software suelen empeorar la latencia base del hardware en los sistemas distribuidos [3]:

a) **Garantías ACID:** Para evitar pérdida de datos, los SGBD fuerzan escrituras síncronas (`fsync()`). En operaciones de un solo hilo, la velocidad del sistema se amarra por completo a la lentitud individual del disco.

b) **Concurrencia (Tail Latency):** Aumentar peticiones en paralelo sube el rendimiento global (IOPS), pero incrementa la latencia promedio y de cola, retrasando la respuesta de los hilos.

c) **Almacenamiento Web:** APIs síncronas como `LocalStorage` corren en el hilo principal (*main thread*); si se guardan datos pesados, la interfaz se congela. Esto se mitiga usando `IndexedDB`, que gestiona los datos de forma asíncrona sin bloquear la pantalla.

3. Comparativa Sintética de Rendimiento y Arquitectura

La tabla resume las características físicas y lógicas de los diferentes niveles de almacenamiento y su impacto en el rendimiento al momento de acceder a fragmentos de datos [1].

Dimensión / Parámetro	Discos Mecánicos (HDD)	Estado Sólido (SSD)	Discos NVMe (PCIe)	Memoria RAM
Mecanismo / Ubicación	Platos magnéticos y cabezal mecánico.	Celda Flash con controlador SATA.	Celda Flash sobre bus PCIe directo.	Memoria Principal (Estructura Compacta).
Latencia Típica (Escritura 4K)	47,85 ms (milisegundos)	145,53 μs (microsegundos)	Menor a 100 μs (multicola)	100.000x más rápida que el disco
Escritura 5 GB (`dd`)	24,86 s (216 MB/s)	12,56 s (427 MB/s)	4,93 s (1.100 MB/s)	Sin operaciones E/S a disco
Consumo y Rendimiento	Alto costo en accesos aleatorios.	Alto rendimiento en IOPS paralelas.	Máxima concurrencia y ancho de banda.	Solo 38 % de espacio vs. S2I (62 % ahorro).

4. Mitigación: Estructuras Compactas y Almacenamiento en Crudo

Para suprimir el costo de las operaciones de E/S a disco, el diseño arquitectónico de bases de datos modernas aplica dos soluciones principales [5]:

- **Estructuras Compactas en RAM (cBiK):** Representan datos espacio-textuales masivos mediante árboles iKD-Tree y bitmaps sucintos. Frente a índices en disco como S2I, reducen el uso de memoria al 38 % y resuelven consultas hasta 3 órdenes de magnitud más rápido al eliminar los accesos a almacenamiento secundario.
- **Backends en Crudo (BlueStore):** Acceden directamente al dispositivo de bloques omitiendo los sistemas de archivos locales POSIX. Al gestionar metadatos de forma asíncrona mediante *direct I/O*, eliminan la penalización del doble journaling.

5. Tres Ejemplos Prácticos de la Influencia de la Latencia en el Software

Se presentan escenarios prácticos donde el diseño, la ubicación de los datos y la latencia afectan directamente la experiencia del usuario [4]:

- **SGBD Relacionales (Banca):** Las transacciones síncronas en discos mecánicos (HDD) limitan físicamente el rendimiento a $\sim$66 operaciones por segundo. Migrar a hardware NVMe disminuye drásticamente el tiempo de bloqueo, escalando a miles de IOPS.
- **Aplicaciones Web (E-Commerce):** Usar `LocalStorage` de forma síncrona en el navegador causa micro-congelamientos (*jank*). Sustituirlo por `IndexedDB` procesa los datos en segundo plano asíncronamente, manteniendo la interfaz fluida a 60 FPS.
- **Consultas Geográficas (Movilidad):** Buscar vehículos cercanos en índices tradicionales de disco exige múltiples lecturas aleatorias. Al migrar a estructuras compactas en memoria RAM (cBiK), las consultas se resuelven sin descompactar la información, reduciendo el tiempo de respuesta hasta 1.000 veces.

6. Conclusiones

- **L**a brecha de rendimiento físico entre la memoria principal y el almacenamiento secundario representa el principal cuello de botella en la velocidad de respuesta del software. Para mitigar esta latencia, no solo se requieren estructuras de datos compactas y hardware de estado sólido (NVMe), sino que, a nivel arquitectónico, el diseño de las bases de datos distribuidas debe aplicar técnicas de fragmentación y asignación óptima para acercar la información al usuario, reduciendo el costo de las operaciones de red y escalando eficientemente [1, 2].
- **L**as garantías transaccionales síncronas y el almacenamiento bloqueante en clientes web (como LocalStorage) amplifican la latencia física de forma crítica. Es imprescindible adoptar operaciones de E/S asíncronas e índices orientados a la distribución de datos, los cuales previenen micro-congelamientos de la interfaz, aseguran una alta disponibilidad mediante el procesamiento concurrente y mejoran el tiempo de respuesta global del sistema [3, 4, 5].

Referencias

[1] Y. Alfaro Abreus, "Tipos de fragmentación en bases de datos distribuidas," *Ingeniería Industrial*, vol. 27, no. 1, pp. 3–6, 2006.

[2] E. L. Iglesias, "Bases de datos distribuidas," ESEI, Univ. de Vigo, España, 2005. [En línea]. Disponible en: `http://trevinca.ei.uvigo.es/~evali/abd/BDDWeb.pdf`

[3] A. Hernández González, "Base de datos distribuida," Tesis de maestría, CEIS, Ciudad de La Habana, Cuba, 2005.

[4] Cinvestav, "Diseño de Bases de datos distribuidas," 2005. [En línea]. Disponible en: `http://www.cs.cinvestav.mx/SC/prof_personal/adiaz/Disdb/Cap_3.html`

[5] J. Rodríguez Santos, "Diseño de bases de datos distribuidas," 2005. [En línea]. Disponible en: `http://usuarios.lycos.es/jrodr35/index.htm`

Declaración de Uso de Inteligencia Artificial

Para la elaboración del presente documento se empleó la herramienta **NotebookLM** con el propósito principal de sintetizar, abstraer e interpretar la información técnica y teórica analizada en los documentos base. Adicionalmente, se utilizó el modelo de lenguaje **Gemini** como asistente de soporte en la redacción, corrección de estilo y generación del código estructurado en formato LaTeX, garantizando su óptima maquetación y compilación dentro de la plataforma Overleaf.